

Per%Sense Port — Five Things We Need From You

Date: 2026-06-11

The Go port now matches the original DOS engine to the last displayed digit across essentially every calculation, verified automatically against the original code. In getting the final fraction of confidence nailed down, we ran into a small number of points that aren't ours to decide — they're either missing materials or judgment calls about *which* behavior you want to be the standard. None of these affect a normal, everyday calculation. Here they are, shortest first.

1. The actuarial (life-contingency) source — the one real blocker

The life-contingency feature (life tables, payment-on-death, two-life contingencies) was built, but its core computation lived in a separate code unit named **ACTUARY** that **wasn't included** in the source we received, and the feature was switched off in the shipped DOS build. We've reconstructed it and verified its math against independent actuarial standards (it agrees to nine-plus decimals), so we're confident it's *correct* — but we can't prove it matches *your original's exact numbers* without the original materials.

What would unblock it (any one): the `ACTUARY.PAS` source, an executable of Per%Sense with life-contingency turned on, or the mortality table it used (name/ year, e.g. the 1988 HHS table) plus the manual's worked examples.

Question: Can you locate any of those?

2. Present-value screen — a place where our port is a bit *more* forgiving

On the Present Value screen, our port lets a user type a single payment's value in that row's own **Value** column and solve from it. The original required the target to go in the screen's **Sum Value** line and otherwise refused. The computed numbers are identical — only *which input layout is accepted* differs.

Question: Keep the more forgiving behavior (we'd document it as an improvement), or tighten it to match the original exactly? **Our recommendation:** keep it — it's a convenience, and no result changes.

3. An amortization edge case where the *original itself* misbehaves

If someone puts an adjustable rate (ARM) **and** a balloon (or skip-months / target) on the **same** loan, the original DOS engine doesn't converge: after the rate change it drops the payment to almost nothing, lets the balance grow, and clears it all with one giant final payment. Our port instead produces a clean, normal amortization. So matching the original here would mean deliberately copying a malfunction, on an uncommon input combination.

Question: Is "an ARM and a balloon on one loan" in scope, and if so do you want us to reproduce the original's behavior exactly (malfunction and all) or keep our cleaner result? **Our recommendation:** keep our behavior; it's the more useful result.

4. A day-count detail on an unusual basis setting

On the **actual/365** day-count basis with **monthly** payments (an uncommon combination), the original charges the *first* month's interest using the simple "rate ÷ 12," while our port prorates it by the actual number of days. The difference is tiny (first month only, ~0.008% of the loan) and only appears on that one basis setting — the normal 30/360 monthly case matches exactly.

Question: Keep our actual-day first-month interest (more consistent with an "actual days" basis), or match the original's "rate ÷ 12"? **Our recommendation:** keep ours; no common loan changes.

5. Ads / policy note (only if relevant)

(Skip unless it has come up.) No action needed — included only so the list is complete if you had questions about distribution.

Bottom line: Item 1 (the actuarial materials) is the only one that blocks further verification; the other three are short "which behavior do you prefer" decisions where no normal calculation changes either way. A yes/no on each is all we need.